ASP.NET Core In-Process Hosting Model

When we build a web application in ASP.NET Core, it requires a **Runtime Environment** to receive HTTP requests, process them, and send responses back to clients. A **Web Server** and a **Hosting Model provide this runtime environment**. There are two hosting models: In-Process and Out-of-Process. **In-process hosting** plays a significant role in boosting performance by directly running the application within the IIS worker process.

Now, we will explore **In-Process Hosting in ASP.NET Core** in detail, starting from the basics of web servers and hosting, moving to different hosting models, understanding how the In-Process model works step by step, and finally configuring it in a real ASP.NET Core Web API application.

What is a Web Server?

A **Web Server** is a specialized software that listens for **HTTP/HTTPS requests** from clients (such as browsers, mobile apps, or API clients), processes them, applies built-in rules (like TLS/SSL termination, connection limits, request size enforcement, compression), and then hands valid requests over to your **application code** for execution. Once the application generates a response (HTML, JSON, files, etc.), the web server sends that response back to the client efficiently and securely.

For example:

- A client requests https://myapi.com/products.
- The web server accepts the request, validates it, and forwards it to the application.
- The application executes the required logic, and the web server delivers the response back to the client who made the original request.

Why Do We Need a Web Server?

We need a web server because:

Acts as a Bridge between clients and our application code.

- Clients cannot directly execute application code.
- The web server translates HTTP requests into a format the application can handle and returns structured responses.

Manages Connections, Protocols, and Request Lifecycles.

- Manages TCP connections, as well as HTTP/1.1, HTTP/2, and HTTP/3.
- Controls request pipelines from start (receive request) to finish (send response).

Enforces Resource and Request Rules.

- Applies connection limits, request size restrictions, and timeout rules.
- Protects applications from overload, malformed requests, or denial-of-service attacks.

Provides Advanced Features for Production.

- **Process Management:** Start, stop, and recycle worker processes automatically.
- **Security:** SSL/TLS termination, request filtering, authentication/authorization hooks.
- **Performance Optimizations:** Response caching, compression.
- **Load-Balancer Integration:** Works with reverse proxies or hardware load balancers.
- **Logging & Monitoring:** Captures request/response logs, error reports, and diagnostics.

On Windows and Cross-Platform

On **Windows**, the most common web server is Internet Information Services (IIS**)**.

- IIS is a full-featured enterprise web server with process supervision, logging, and rich configuration options.

In **ASP.NET Core**, there is also **Kestrel**, a high-performance cross-platform web server embedded in your application.

- Kestrel is always present in ASP.NET Core apps.
- It can run **standalone** (direct client communication) or **behind a reverse proxy** like IIS, Nginx, or Apache.

Real-Time Analogy

Think of a **Web Server** as the **Reception Desk of a Corporate Office**:

- Visitors (clients) arrive at the reception (web server).
- The receptionist checks IDs, enforces security policies, and ensures rules are followed (including timeouts, request size, and authentication).
- If everything is valid, the receptionist passes the visitor's request to the right employee (application code).
- The employee prepares the required information (response).
- The receptionist hands it back to the visitor neatly.

Without the receptionist, employees would be distracted and burdened with tasks such as verifying IDs, enforcing rules, or handling interruptions, rather than focusing on their actual work. Similarly, a web server shields the application from direct client communication and ensures that requests are processed in a safe, efficient, and manageable manner.

What is Hosting?

Hosting means running an application inside a **managed environment** (called a **Host**) that makes it available to serve real client requests.

Without hosting, your ASP.NET Core project is simply a set of compiled **DLLs, configuration files, and code**. On its own, that code cannot respond to HTTP requests. The host is what turns your code into a **live, running web application**.

In ASP.NET Core:

- Applications are **not standalone executables** in the traditional sense.
- They always need a **Host** that sets up the environment, manages dependencies, configures logging, creates a request pipeline, and controls the lifetime of the app.

Why Do We Need Hosting?

- Without a host, your web app would be code sitting on disk.
- Hosting is the process of **deploying an application onto a machine/server** and making it **accessible to the outside world over the internet**.
- It also ensures the app runs in a **predictable, stable, and secure runtime environment** (with proper logging, configuration, error recovery, etc.).

In simple words: Hosting is the **bridge between your code and the real-world clients** who want to use it.

Responsibilities of Hosting

Hosting is much more than just “starting the app.” It ensures the app is **alive, stable, and capable of handling requests** throughout its lifetime. A Host:

Keeps the Application Alive and Running

- Monitors application lifetime.
- Restarts it if it crashes (when used with IIS, Docker, etc.).

Accepts Incoming HTTP Requests

- The host sets up the web server (Kestrel or IIS integration).
- It listens for client requests on configured ports (e.g., https://localhost:5000).

Passes Requests to the ASP.NET Core Request Pipeline

- Requests are translated into an HttpContext.
- Middleware components process authentication, authorization, routing, etc.
- Finally, the request reaches your **controllers or minimal APIs**.

Returns Responses Back to Clients

- The host ensures responses are serialized and sent back properly.
- Also applies cross-cutting concerns like compression, logging, and error handling.

Manages Application Startup & Lifetime

- Reads appsettings.json, environment variables, and command-line arguments.
- Registers services into the DI container.
- Executes startup logic (Program.cs).
- Cleans up resources when the application shuts down.

Real-time Analogy:

Think of **hosting** like **opening a restaurant**:

- Your **chefs, recipes, and kitchen equipment** = Your application code and DLLs.
- But recipes on paper don't serve customers by themselves. Until you **rent a space, set up tables, open the doors, and hire staff to serve customers** (the host environment), your restaurant is not "running."
- You need a **restaurant (the host)** to:
  - Keep the doors open (keep the app running).
  - Have staff ready to welcome guests (accept HTTP requests).
  - Route each guest to the right waiter and chef (middleware pipeline + controllers).
  - Serve food back to the table (return responses).
  - Manage lights, bills, cleaning, and closing (lifetime management, logging, shutdown).

Without the restaurant, the recipes are useless to the outside world. Similarly, without hosting, your code cannot serve real users.

Hosting Models in ASP.NET Core

When you deploy an ASP.NET Core application, the **hosting model** determines how the application runs in relation to the web server. The two main models are **In-Process** and **Out-of-Process**.

In-Process Hosting Model

In the **In-Process Hosting Model**, the ASP.NET Core application runs directly inside the IIS worker process (w3wp.exe for IIS or iisexpress.exe for IIS Express). IIS does not forward requests to a separate process. Instead, the request pipeline is integrated within the worker process using IISHttpServer. Because there is no inter-process communication, this model delivers the best performance on Windows servers.

Key Points:

- Application runs inside the **IIS worker process (w3wp/iisexpress)**.
- Uses **IISHttpServer** instead of Kestrel.
- No reverse proxying, no additional hops.
- Offers **higher performance and lower latency**.
- Only available on **Windows with IIS/IIS Express**.

Analogy:

Think of In-Process hosting like a **chef cooking directly in the dining area**. Customers hand orders straight to the chef, and food is prepared immediately without going through any middleman. It's fast and efficient, but only works in setups where the kitchen and dining area are tightly integrated (like IIS on Windows).

Out-Of-Process Hosting Model

In the **Out-of-Process Hosting Model**, the ASP.NET Core application runs in a **separate process** (Kestrel server worker process). IIS (or another reverse proxy, such as Nginx or Apache) receives client requests and then forwards them to the application via Kestrel. The app processes the request and sends the response back through the proxy. This additional communication step introduces a slight performance overhead but provides greater flexibility and cross-platform compatibility.

Key Points:

- Application runs in a **Separate Process** (dotnet.exe or MyApp.exe).
- **Kestrel web server** handles requests inside that process.
- IIS (or IIS Express or Nginx/Apache) acts as a **reverse proxy** to forward traffic to Kestrel.
- Introduces a **proxying overhead**, slightly slower than InProcess.
- Works on **Windows, Linux, and macOS**.
- Necessary for **Cross-Platform Hosting** (Linux/macOS) or containerized environments.
- Slightly slower than In-Process due to proxying overhead.

- Commonly used with **Nginx/Apache reverse proxy** on non-Windows systems.

Analogy:

Think of Out-of-Process hosting like a **restaurant with waiters**. Customers place orders with the waiter (IIS/Nginx/Apache), the waiter carries the order to the chef (Kestrel), and then brings the food back to the customer. The process is slightly slower than handing orders directly to the chef, but it enables better service, organization, and flexibility across various environments.

Hosting Models by Platform

ASP.NET Core applications can be hosted differently depending on the platform. On **Windows**, IIS (or IIS Express during development) is often used. On **Linux/macOS**, apps typically run with Kestrel, often behind a reverse proxy such as Nginx or Apache.

Hosting on Windows (IIS)

On Windows, ASP.NET Core applications are commonly hosted behind **IIS (Internet Information Services)**. IIS integrates deeply with Windows, Handling Connections, SSL Termination, Process Management, and Logging.

When you use IIS, you can choose between the **In-Process model** (faster, app runs inside the IIS worker process) or the **Out-of-Process model** (IIS acts as a reverse proxy to a separate Kestrel process). IIS simplifies deployment in enterprise Windows environments by providing advanced management features and security options.

Key Points:

- Supports both **In-Process** (where the app runs inside w3wp.exe / iisexpress.exe) and **Out-of-Process** (IIS → Kestrel) scenarios.
- IIS provides **Windows Authentication**, request filtering, logging, and process recycling.
- Best suited for **enterprise Windows environments** that require IIS features.

Real-time Analogy:

Hosting on Windows with IIS is like running a **restaurant in a hotel**: sometimes the chef works right in the hotel kitchen (In-Process), other times the hotel staff sends orders to a separate partner kitchen down the street (Out-of-Process). In both cases, the hotel manages the front desk and ensures guests are served properly.

Hosting on Linux/macOS

On Linux and macOS, IIS is not available. Instead, ASP.NET Core apps run on **Kestrel** and are typically placed behind a **Reverse Proxy Server** such as **Nginx** or **Apache**. The reverse proxy handles edge concerns, such as SSL Termination, Load Balancing, and Static File Caching, while Kestrel focuses on serving ASP.NET Core

requests. In containerized environments (like Docker), you can also run Kestrel directly without a reverse proxy, though a proxy is recommended for production to improve security and manageability.

Key Points:

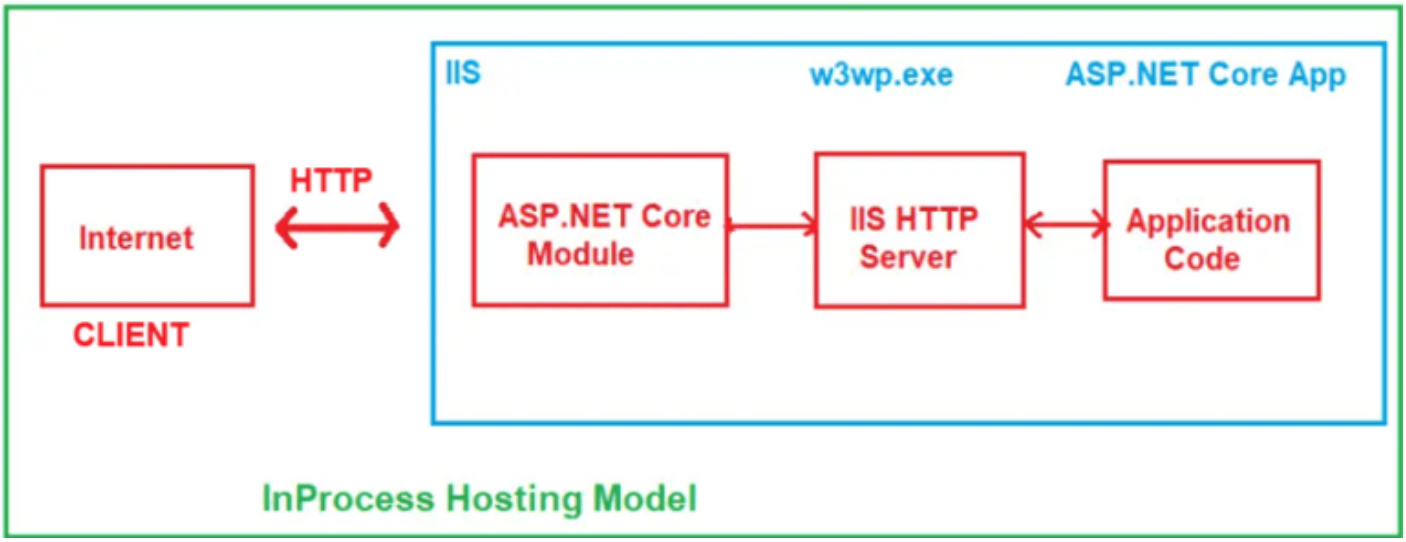
- **Kestrel is the primary web server** — runs the ASP.NET Core application directly.
- Common setup: **Kestrel + Nginx/Apache** (reverse proxy handles SSL, routing, static files).
- Supports **self-hosting** in containers (Kestrel only).
- Flexible, cross-platform, lightweight, and high-performance.
- Recommended to use a **reverse proxy in production** for security, scalability, and resilience.

Real-time Analogy:

Hosting on Linux/macOS is like setting up a **food truck**: Kestrel is the chef in the truck, cooking meals directly. If it's parked in a busy street, you'll often see a **front counter manager (Nginx/Apache)** handling payments, crowd control, and packaging before handing the food to customers. In a private event (such as a container or internal service), the chef may serve directly without a counter manager.

How does the In-Process Hosting Model Work in ASP.NET Core?

The **In-Process Hosting Model** is used when an ASP.NET Core application runs **inside the IIS worker process (w3wp.exe or iisexpress.exe)**. This removes the need for proxying requests to Kestrel, resulting in better performance. Please look at the following diagram to better understand how the In-Process Hosting Model works with an ASP.NET Core Application.



The InProcess Hosting Model works as follows:

Step 1: User Request

The process starts when a client (like a browser or mobile app) sends an HTTP/HTTPS request to your web application. For example, the user might enter a URL (https://myapp.com/products) or make an API call. This request travels over the internet and reaches the web server (IIS) that is hosting the ASP.NET Core application.

Responsibilities:

- Client (browser/app) prepares and sends an HTTP request.
- The request includes the URL, HTTP method (e.g., GET, POST), headers, and possibly the body.
- Requests travel over the Internet until they reach the IIS server.
- IIS (on the server) is the first to receive this request.

Layman Example: Think of a **customer walking into a restaurant** and placing an order ("I want a pizza").

Mapping:

- **Client** → Customer
- **Request** = Customer order.
- **Internet** = Road the customer uses to reach the restaurant.
- **IIS** = Reception desk at the restaurant, the first point of contact.

Step 2: IIS (Internet Information Services)

IIS acts as the entry point into the hosting environment. It receives the request and checks if the ASP.NET Core app is already running inside its worker process (w3wp.exe).

- If the app is not yet running, IIS uses the **ASP.NET Core Module (ANCM)** to start the app.
- If the app is already running, IIS skips ANCM and passes the request directly to the in-process HTTP server **(IIS HTTP Server)**.

Responsibilities:

- Receives incoming HTTP request.
- Checks if the application pool (worker process) is active.
- If app is not running → asks ANCM to start it.
- If the app is running → forwards the request to the IIS HTTP Server.

Layman Example: At the restaurant, the **receptionist checks** whether the kitchen is open. If the kitchen is closed, the receptionist calls the chef to get things started. If the kitchen is already running, the order is sent straight to the chef's assistant.

Mapping:

- **IIS** = Receptionist.
- **Application Pool Check** = Confirming if the Kitchen is open.
- **ANCM** = Call to the chef to open the kitchen.
- **Forwarding to the IIS HTTP Server** = Passing the order to the chef's assistant.

Step 3: ASP.NET Core Module (ANCM)

ANCM is a native IIS component that is responsible for starting and loading the ASP.NET Core application into IIS's worker process. It doesn't participate in every request; its primary role is **initialization**. When triggered, it loads the .NET Core runtime, initializes the app inside the IIS worker process, and hands control to the IIS HTTP Server.

Responsibilities:

- Loads the .NET Core runtime into w3wp.exe.
- Start the ASP.NET Core application within an IIS worker process.
- The hands request processing to the IIS HTTP Server (the in-process request handler).
- After startup, it no longer intercepts requests.

Layman Example: If the kitchen is closed, the receptionist (IIS) calls the **kitchen manager (ANCM)**. The manager turns on the lights, brings the chef and staff, and sets up the kitchen so it's ready to cook. After that, the manager steps aside.

Mapping:

- **ANCM** = Kitchen manager.
- **Loading .NET runtime** = Setting up the kitchen.
- **Starting the app** = Chef arriving and preparing tools.
- **Handing control to the IIS HTTP Server** = Passing the order to the assistant chef.

Step 4: IIS HTTP Server

The IIS HTTP Server (IISHttpServer) is the in-process HTTP handler that bridges IIS and ASP.NET Core. It takes the native IIS Request, parses the request (method, headers, protocol), and translates it into a .NET **HttpContext** object that ASP.NET Core middleware understands. Then, it forwards the request to the application pipeline.

Responsibilities:

- Receives request from IIS/ANCM.
- Decodes HTTP details (method, URL, headers, body).
- Creates an HttpContext object for ASP.NET Core.
- Forwards request to ASP.NET Core middleware pipeline.

Layman Example: Think of the **assistant chef** who takes the customer's order slip and rewrites it in a way the chef understands (converting the customer's words into kitchen terms).

Mapping:

- **IIS HTTP Server** = Assistant chef.
- **Parsing HTTP** = Translating the order.
- **HttpContext** = Order slip formatted for the kitchen.
- **Forwarding to middleware** = Handing it to the chef.

Step 5: Application Code (ASP.NET Core App)

This is where the real business logic runs. The request enters the **ASP.NET Core middleware pipeline**, which can include Authentication, Authorization, Routing, Logging, and Error Handling. Finally, the request reaches the correct Controller or minimal API endpoint, where the actual business logic runs, and a response (in JSON, HTML, or file format) is prepared.

Responsibilities:

- Execute middleware pipeline (auth, routing, logging, exception handling, etc.).
- Perform routing to find a matching endpoint/controller.
- Run application code (controller action, services, DB calls).
- Creates a response object (e.g., JSON, HTML).

Layman Example: The **chef** receives the order slip, prepares the dish, adds seasoning (extra logic), and completes the meal.

Mapping:

- **Middleware pipeline** = Preparation steps (cleaning, chopping, cooking sequence).
- **Controller Action** = Chef making the dish.
- **Response** = Finished meal.

Step 6: IIS HTTP Server (Return Path)

After the app prepares the response, it sends the completed response back to the IIS HTTP Server. This server finalizes the response, serializes it correctly into HTTP format, and returns it to IIS, ready to be sent over the network.

Responsibilities:

- Receives response from application code.
- Convert ASP.NET Core response into IIS-native format.
- Apply any final IIS-level handling (e.g., response buffering).
- Passes it back to IIS.

Layman Example: The **assistant chef** takes the completed dish from the chef, plates it properly, and hands it back to the receptionist.

Mapping:

- **App response** = Finished dish.
- **IIS HTTP Server** = Assistant chef plating and packaging the meal.
- **Forwarding to IIS** = Returning the dish to the receptionist.

Step 7: IIS

Finally, IIS sends the completed response back over the Internet to the client. The client (browser, app) receives it and presents it to the user.

Responsibilities:

- Receives response from the IIS HTTP Server.
- IIS logs the transaction for monitoring.
- Sends response back to the client over HTTP/HTTPS.
- Ends the request lifecycle.

Layman Example: The **receptionist** hands the plated meal to the customer. The customer leaves satisfied.

Mapping:

- **IIS** = Receptionist handing food.
- **Response** = Meal served to customer.
- **Client** = Customer eating the meal.

Final Recap

- **Client** → Customer arriving.
- **IIS** → Receptionist.
- **ANCM** → Kitchen manager (only at startup).
- **IIS HTTP Server** → Assistant chef (translator + plating).
- **ASP.NET Core App** → Chef preparing the dish.

- **Response Flow** → Meal going back via assistant → receptionist → customer.

Example to Understand In-Process Hosting Model in ASP.NET Core Web API:

The following TestController is a simple **Web API controller** that exposes one endpoint **/api/test/ping**. When you hit this endpoint, it collects and returns diagnostic information about the **hosting environment** of your ASP.NET Core application. Specifically, it shows:

- The **Process ID** of the running application.
- The **Process name** (e.g., w3wp for IIS InProcess, iisexpress for IIS Express, dotnet or exe name for OutOfProcess/Kestrel).
- The **Server Type** (which web server is executing requests — IISHttpServer for InProcess, KestrelServer for OutOfProcess).
- The **Machine Name** (server name where the app is running).

This way, we can confirm whether our app is running **InProcess** (inside the IIS worker process) or **OutOfProcess** (in a separate Kestrel process). So, create an API Empty controller named **TestController** within the **Controllers** folder, and then copy and paste the following code.

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Hosting.Server;
using System.Diagnostics;

namespace MyFirstWebAPIProject.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class TestController : ControllerBase
    {
        // The IServer interface represents the actual server implementation
(IISHttpServer or KestrelServer)
        private readonly IServer _server;

        // Constructor Dependency Injection - ASP.NET Core injects the active
IServer service
        public TestController(IServer server)
        {
            _server = server;
        }

        // GET: api/test/ping
        [HttpGet("ping")]
        public IActionResult Ping()
        {
```

```
            // Get the current process information (e.g., w3wp, iisexpress,
dotnet)
            var proc = Process.GetCurrentProcess();

            // Return hosting environment details as JSON response
            return Ok(new
            {
                message = "Hosting Info",               // Custom message
                pid = Environment.ProcessId,            // Numeric process ID
                process = proc.ProcessName,             // Running process name
                serverType = _server.GetType().Name,    // Hosting server type
(IISHttpServer or KestrelServer)
                machine = Environment.MachineName       // Server machine name
            });
        }
    }
}
```

What is IServer?

- IServer is an **interface** in ASP.NET Core (Microsoft.AspNetCore.Hosting.Server) that represents the **HTTP server implementation** used by your application.
- ASP.NET Core is flexible and can run on multiple server implementations. The runtime injects the appropriate server via **Dependency Injection (DI)**.

Use of IServer

- It allows you to **identify** the server handling your requests programmatically:
  - IISHttpServer → when running InProcess under IIS.
  - KestrelServer → when running OutOfProcess under IIS or directly with Kestrel.
- Helps in **diagnostics and debugging**.
- Rarely used in application business logic, but very useful when building **infrastructure code** or middleware that depends on the hosting environment.

How to Configure In-Process Hosting in ASP.NET Core?

By default, when running with **IIS Express** in Visual Studio, ASP.NET Core applications use the **InProcess hosting model**. But sometimes you may want to configure or verify it explicitly. ASP.NET Core offers two primary methods for configuring the hosting model.

Method 1: Using Project Properties File

You can configure the hosting model directly inside the **project file (.csproj)**. This is the most explicit way and works across all environments (not just Visual Studio).

1. Right-click your project in **Solution Explorer** → choose **Edit Project File**.
2. Add the <AspNetCoreHostingModel> element inside <PropertyGroup>.

This setting only applies when hosting in **IIS/IIS Express**. If you run your app directly with' dotnet run', it always uses **Kestrel** (no InProcess option is available).

Once you open the Application Project file, modify it as shown below. As you can see, here we have added the **<AspNetCoreHostingModel>** element and set its value to InProcess.

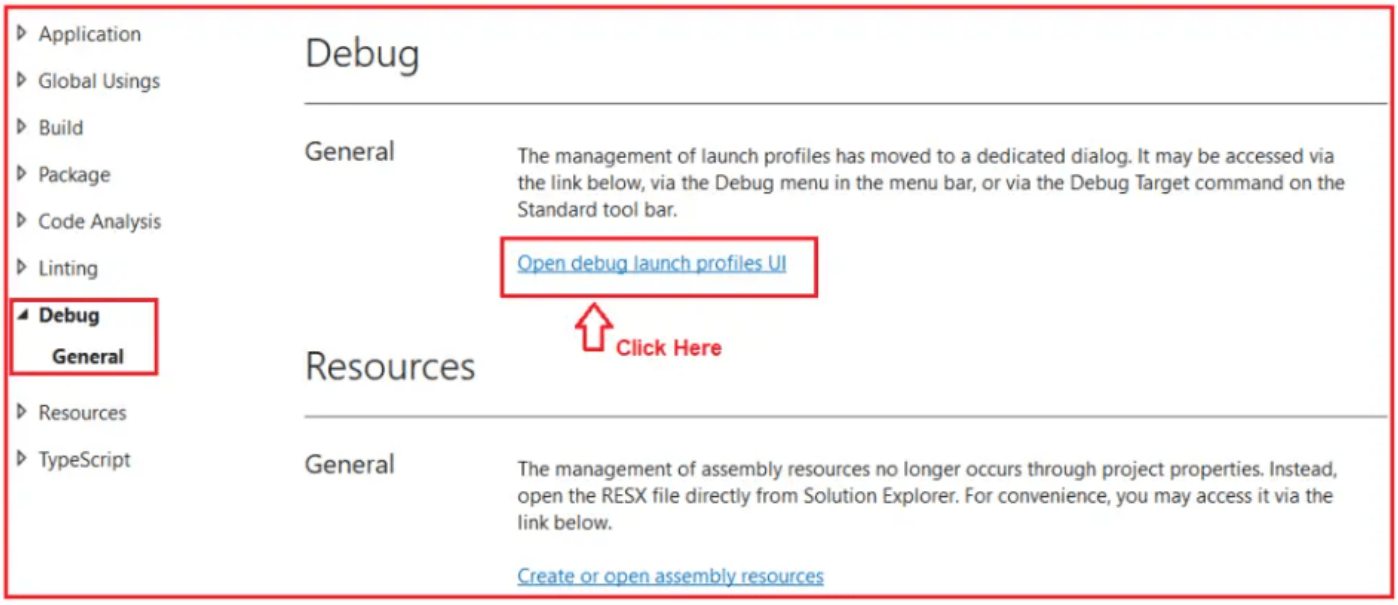
```
<Project Sdk="Microsoft.NET.Sdk.Web">
 <PropertyGroup>
  <TargetFramework>net8.0</TargetFramework>
  <Nullable>enable</Nullable>
  <ImplicitUsings>enable</ImplicitUsings>
  <AspNetCoreHostingModel>InProcess</AspNetCoreHostingModel>
 </PropertyGroup>

 <ItemGroup>
  <PackageReference Include="Swashbuckle.AspNetCore" Version="6.6.2" />
 </ItemGroup>
</Project>
```

Possible Values:

- **InProcess** → App runs inside IIS worker process (w3wp.exe or iisexpress.exe).
- **OutOfProcess** → IIS works as a reverse proxy, and your app runs in dotnet.exe or a self-contained exe.

Method 2: Specify the Hosting Model as In-Process using the GUI

To do so, right-click on your project in the Solution Explorer and select the Properties option from the context menu, which will open the Project Properties window. From this window, select the **Debug tab** and click the **Open Debug Launch Profiles UI** button, as shown in the image below.



Once you click the **Open Debug Launch Profiles UI** button, the Launch Profile window will open. From this window, select **IIS Express** and scroll down, and somewhere you will find the Hosting Model drop-down list with the following values. Here, you can see that the Default hosting model is In Process, and from here, you can also change the hosting model.

How to use InProcess Hosting to run an Application?

Once configured, you must ensure you are actually running the app in the IIS Express profile.

1. In Visual Studio, select **IIS Express** from the launch dropdown in the top toolbar.
2. Run the application.
3. Visual Studio will launch IIS Express, which hosts your ASP.NET Core app **in-process**.

Testing In-Process Hosting:

To confirm whether the application is indeed running in InProcess mode:

- Run the app.
- Hit the test endpoint (e.g., /api/test/ping from the earlier controller).
- You should see output like:

Once you run the application, access the **api/Test/ping** endpoint, and you should see the following output.

The **In-Process Hosting Model** in ASP.NET Core provides the fastest and most efficient way to run applications on Windows with IIS. Executing directly inside the IIS worker process removes the overhead of inter-process communication and ensures tighter integration with IIS features such as process management, logging, and monitoring. For applications running in IIS environments, this model offers simplicity, performance, and reliability, making it the preferred choice when Windows is the hosting platform.

Introduction to Kestrel (Before Out-of-Process Hosting)

Before learning about the **Out-of-Process Hosting Model**, it's essential to understand the role of the **Kestrel Web Server**.

Kestrel is a **cross-platform, lightweight, and high-performance web server** built into ASP.NET Core. Unlike IIS, which is Windows-only, Kestrel works on **Windows, Linux, and macOS**, making it the backbone of ASP.NET Core's cross-platform hosting capability. In Out-of-Process hosting, IIS (or Nginx/Apache on Linux) acts as a **reverse proxy**, forwarding requests to Kestrel, which then processes them through the ASP.NET Core middleware pipeline.

Simply put:

- In **In-Process**, IIS handles requests directly.
- In **Out-of-Process**, IIS (or another proxy) forwards requests to **Kestrel**, which does the actual request processing.

Understanding Kestrel is therefore essential before moving on to the Out-of-Process model, since it becomes the core web server powering your application.

Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.